

CTF Write-up: Flappy

Category: Reversing/Web • Points: 499

Challenge Overview

The challenge presented a Flappy Bird clone written in Rust and compiled to WebAssembly. At first glance it appeared to be a harmless browser game, but the challenge description hinted that something suspicious was happening behind the scenes.

Target URL:

`http://exp.cybergame.sk:7010`

Reconnaissance

Page Source Inspection

As with most web challenges, the first step was checking the page source. Curling the index page immediately revealed a suspicious HTML comment:

```
<!-- TODO: remove before production flag{n0t_th3_r34l_fl4g_l0l} -->
```

This was clearly a decoy flag intended to distract players.

The page also imported a Rust/WebAssembly module:

```
<script type="module">
  import init from './pkg/flappy_wasm.js';
  (async()=>{await init})();
</script>
```

Additionally, after gameplay a hidden “Score Sync” overlay appeared containing a fake Google login form.

Static Analysis of the WASM Binary

Binary Information

```
WebAssembly (wasm) binary module version 0x1 (MVP)
Size: ~158 KB
```

Strings Extraction

Running `strings` on the WASM binary exposed several interesting artifacts:

```
/assets/app.conf
appencryption_keybad key:
oauthclient_idredirect_urigrant_typeauthorization_codecodeuserAgent
POST/api/v1/oauth/tokenContent-Typeapplication/json
score-syncstyledisplay:flex
flappy_wasm::do_exfil
btoa
```

Key Findings

- The application fetches `/assets/app.conf` during startup.
- A function named `do_exfil` handles credential exfiltration.
- The credentials are sent to `/api/v1/oauth/token`.
- `btoa` is used, indicating base64 encoding.
- The fake login form captures `sc-email` and `sc-pass`.

Disassembly Analysis

The WASM binary was disassembled into WAT format using `wasm2wat`.

Analysis of the `init_modal` closure revealed the following flow:

- Fetch `/assets/app.conf`
- Extract `encryption_key` from JSON
- Read email and password fields
- XOR credentials using a cycling key
- Base64-encode the result with `btoa`
- POST the encoded payload to `/api/v1/oauth/token`

Dynamic Analysis

Fetching app.conf

```
fetch('/assets/app.conf')
  .then(r=>r.json())
  .then(console.log)
```

Response:

```
{
  "oauth": {
    "provider": "google",
    "client_id": "894271536842-k3jhf9x2v7m1n5p4q6r0s8t.apps.googleusercontent.com",
    "client_secret": "G0CSPX-xK9mQ2nRghjkk8jL5dT1wY6uI0",
    "redirect_uri": "http://exp.cybergame.sk:7010/callback"
```

```
},
"app": {
  "name": "Flappy Bird Score Tracker",
  "encryption_key": "8849790b3c85b6ced19ab38ae4048f84219a73b84da6679ef0aa9b90f43c6facb1b07b420dcd958e9"
}
```

The provided `encryption_key` turned out to be a red herring. It was never actually used as the XOR key.

Intercepting the POST Request

After submitting the fake Google login form with:

```
Email: root@root.com
Password: roottoor
```

The captured POST body looked like this:

```
{
  "grant_type": "authorization_code",
  "code": "ISRCNwUg0xQNHhYdMlkAXAtHGlgGRg==",
  "client_id": "894271536842-...",
  "redirect_uri": "http://exp.cybergame.sk:7010/callback",
  "ua": "Mozilla/5.0 ...",
  "res": "2400x1350",
  "ts": 1775286898983
}
```

Known-Plaintext XOR Attack

Since the plaintext credentials were known, the XOR key could be recovered directly:

```
key = ciphertext XOR plaintext
```

Where:

- `ciphertext = base64decode(code)`
- `plaintext = "root@root.com:roottoor"`

```
import base64

code = 'ISRCNwUg0xQNHhYdMlkAXAtHGlgGRg=='
encrypted = base64.b64decode(code)
plaintext = b'root@root.com:roottoor'

key = bytes([encrypted[i] ^ plaintext[i] for i in range(len(plaintext))])
```

```
print(key.decode())
```

```
SK-CERT{y0ur_cr3d3n7i4
```

This revealed only part of the flag because the ciphertext length matched the plaintext length.

To expose additional bytes, a longer password was submitted:

```
Email: root@root.com
```

```
Password: roottoorAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

New encoded payload:

```
ISRCNwUg0xQNHhYdMlkAXAtHGlgGRi0yHnUzch4nLXUxMSgvJh51NnU4PBIKbAIE
```

```
import base64
```

```
code = 'ISRCNwUg0xQNHhYdMlkAXAtHGlgGRi0yHnUzch4nLXUxMSgvJh51NnU4PBIKbAIE'
encrypted = base64.b64decode(code)
plaintext = b'root@root.com:roottoorAAAAAAAAAAAAAAAAAAAAAAAAAAAA'
```

```
key = bytes([encrypted[i] ^ plaintext[i] for i in range(len(encrypted))])
print(key.decode())
```

```
SK-CERT{y0ur_cr3d3n7i4ls_4r3_fl4pping_4w4y}SK-CE
```

The repeated **SK-CE** confirmed the XOR key cycles, meaning the complete flag had been recovered.

SK-CERT{y0ur_cr3d3n7i4ls_4r3_fl4pping_4w4y}

Summary

Step	Finding
HTML Source	Contained a fake decoy flag
WASM Strings	Revealed <code>do_exfil</code> , <code>btoa</code> , and exfiltration endpoints
WAT Disassembly	Confirmed XOR encryption + base64 encoding workflow
app.conf	Provided a fake encryption key
Known-Plaintext Attack	Recovered partial XOR key from known credentials
Longer Credentials	Exposed the complete cycling XOR key and full flag

Final Thoughts

The challenge combined WebAssembly reversing, browser-side malware analysis, and a classic XOR known-plaintext attack in a very elegant way.

The theme fits perfectly: a harmless-looking game secretly “flapping away” the player's credentials, while the flag itself becomes the XOR key used in the attack.